

Stack

Stack

- A stack is linear list in which all additions and deletions are restricted to one end, called top
- If you insert a data series into a stack and then remove it, the order of the data will be reverse. i.e. data input as {5,10,15,20} is removed as {20,15,10,5}
- For this reversing attribute stack is called **LIFO- Last in First out**

Stack



FIGURE 3-1 Stack

Basic Stack Operations

The stack concept is introduced and three basic stack operations are discussed.

- **Push**
- **Pop**
- **Stack Top**

Stack Example

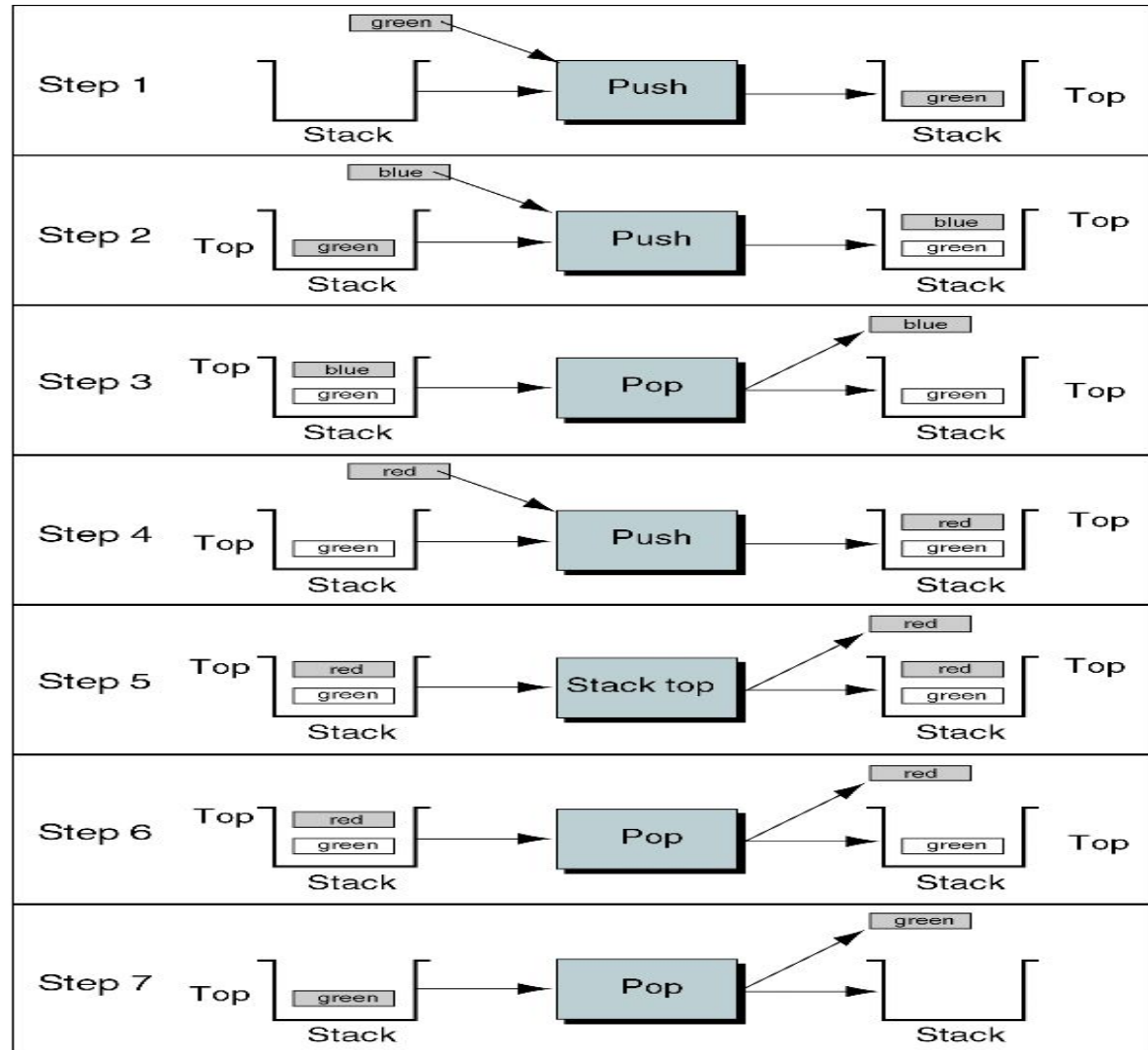


FIGURE 3-5 Stack Example

Stack Linked List Implementation

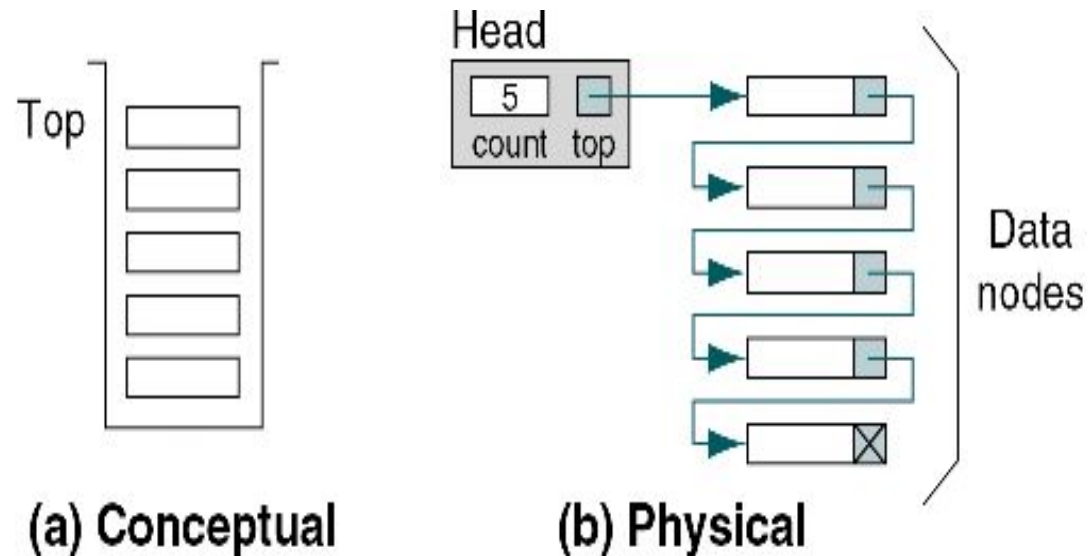
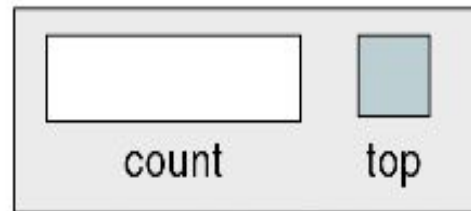
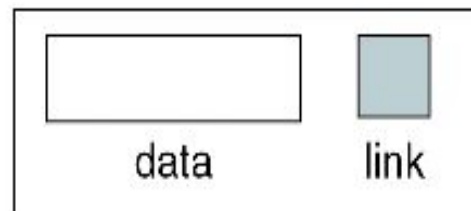


FIGURE 3-6 Conceptual and Physical Stack Implementations

Stack Linked List Implementation



Stack head structure



Stack node structure

```
stack  
  count  
  top  
end stack
```

```
node  
  data  
  link  
end node
```

FIGURE 3-7 Stack Data Structure

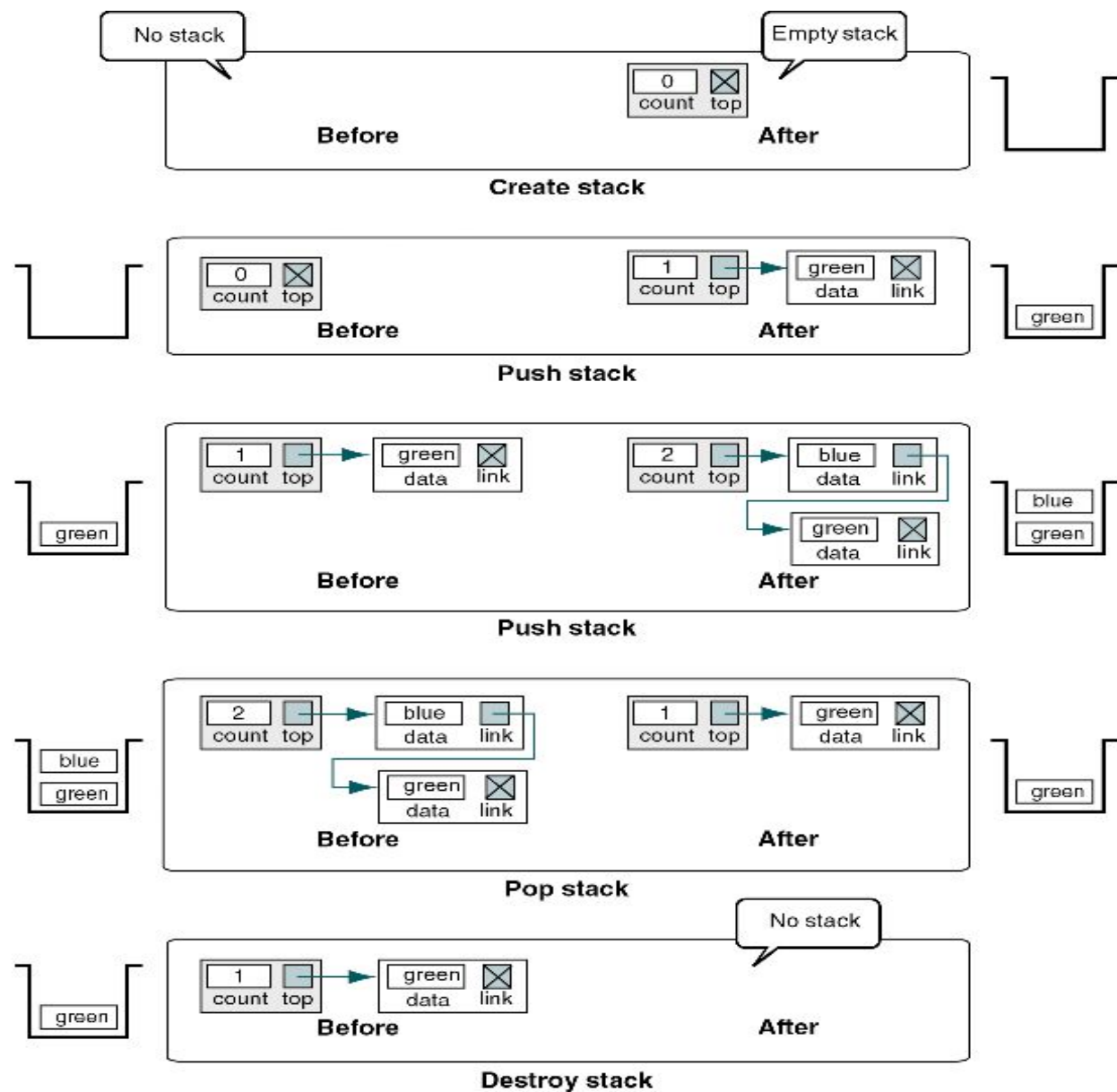


FIGURE 3-8 Stack Operations

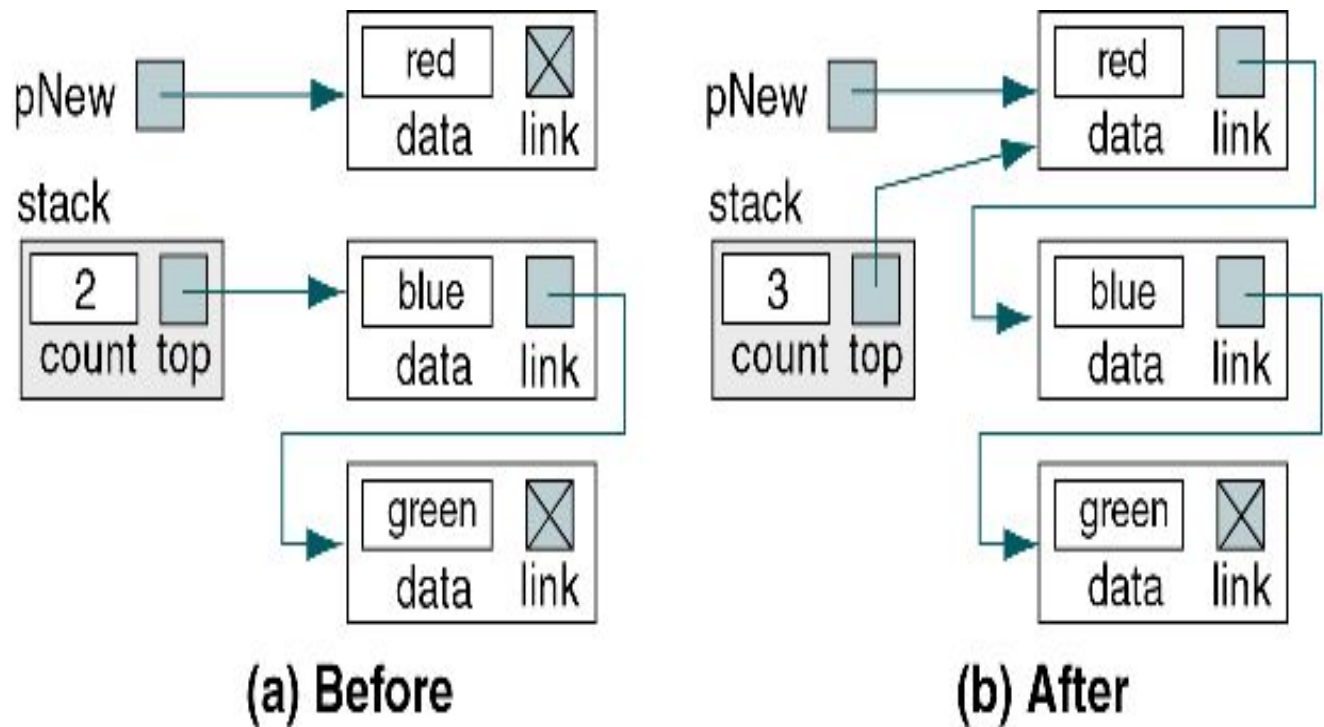


FIGURE 3-9 Push Stack Example

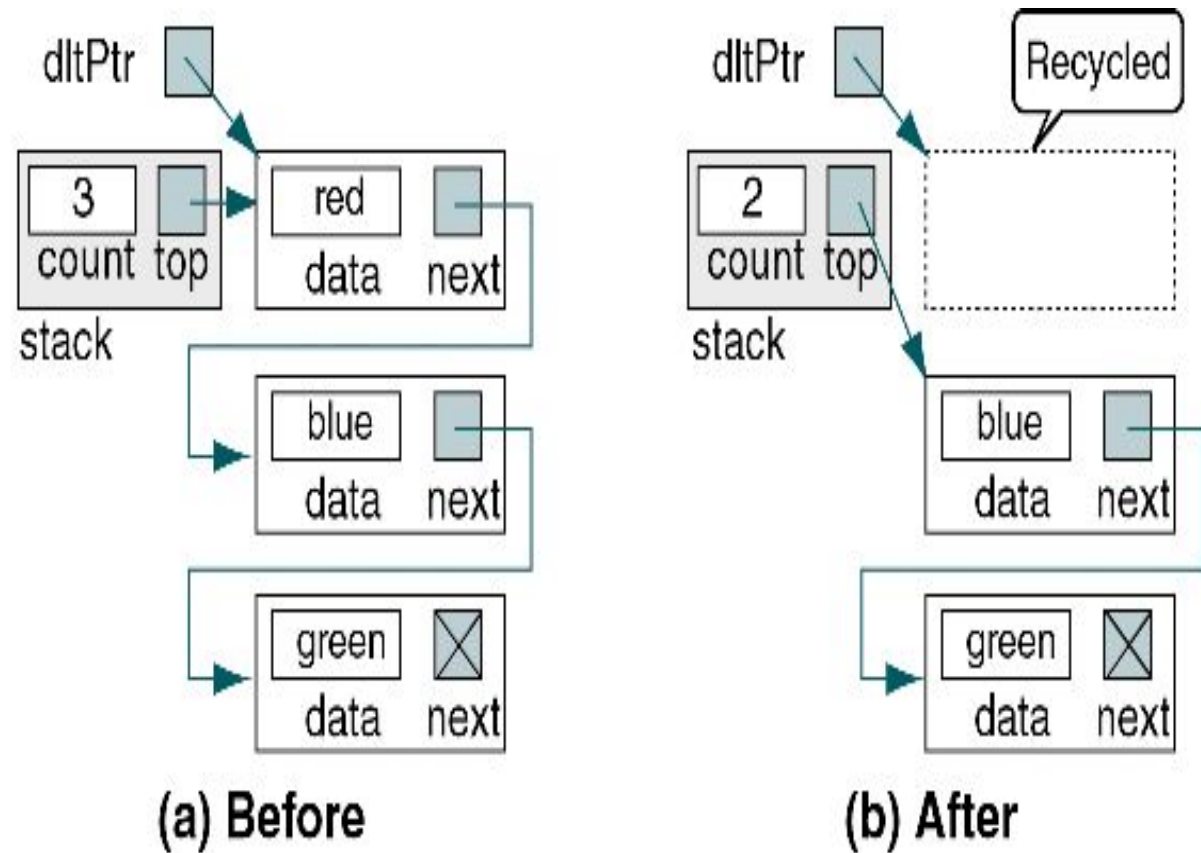


FIGURE 3-10 Pop Stack Example

3-4 Stack ADT

We begin the discussion of the stack ADT with a discussion of the stack structure and its application interface. We then develop the required functions.

- Data Structure
- ADT Implementation

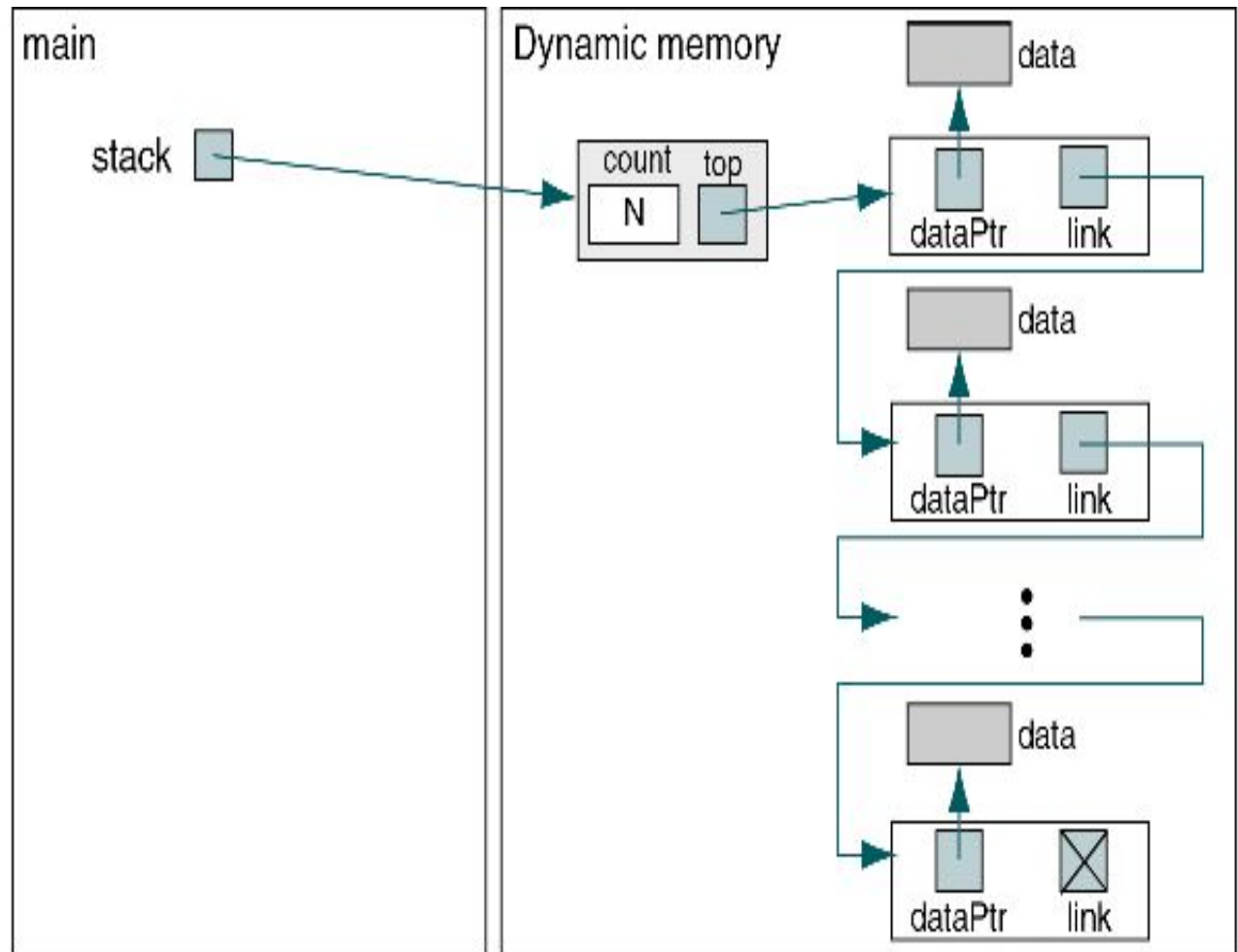


FIGURE 3-12 Stack ADT Structural Concepts

PROGRAM 3-6 Stack ADT Definitions

```
1  // Stack ADT Type Defintions
2  typedef struct node
3  {
4      void*      dataPtr;
5      struct node* link;
6  } STACK_NODE;
7
8  typedef struct
9  {
10     int          count;
11     STACK_NODE* top;
12 } STACK;
```

PROGRAM 3-7 ADT Create Stack

```
1  /* ===== createStack =====
2      This algorithm creates an empty stack.
3      Pre  Nothing
4      Post Returns pointer to a null stack
5              -or- NULL if overflow
6  */
7  STACK* createStack (void)
8  {
9      // Local Definitions
10     STACK* stack;
11
12     // Statements
13     stack = (STACK*) malloc( sizeof (STACK));
14     if (stack)
15     {
16         stack->count = 0;
17         stack->top    = NULL;
18     } // if
19     return stack;
20 }
```

PROGRAM 3-8 Push Stack

```
1  /* ===== pushStack =====
2      This function pushes an item onto the stack.
3      Pre      stack is a pointer to the stack
4                dataPtr pointer to data to be inserted
5      Post     Data inserted into stack
6      Return   true  if successful
7                false if underflow
8  */
9  bool pushStack (STACK* stack, void* dataInPtr)
10 {
11     // Local Definitions
12     STACK_NODE* newPtr;
13
14     // Statements
15     newPtr = (STACK_NODE* ) malloc(sizeof( STACK_NODE));
16     if (!newPtr)
```

PROGRAM 3-8 Push Stack (continued)

```
17         return false;
18
19     newPtr->dataPtr = dataInPtr;
20
21     newPtr->link     = stack->top;
22     stack->top       = newPtr;
23
24     (stack->count)++;
25     return true;
26 } // pushStack
```

PROGRAM 3-9 ADT Pop Stack

```
1  /* ===== popStack =====
2      This function pops item on the top of the stack.
3      Pre  stack is pointer to a stack
4      Post Returns pointer to user data if successful
5           NULL if underflow
6  */
7  void* popStack (STACK* stack)
8  {
9      // Local Definitions
10     void*      dataOutPtr;
```

PROGRAM 3-9 ADT Pop Stack (continued)

```
11     STACK_NODE* temp;
12
13     // Statements
14     if (stack->count == 0)
15         dataOutPtr = NULL;
16     else
17     {
18         temp      = stack->top;
19         dataOutPtr = stack->top->dataPtr;
20         stack->top = stack->top->link;
21         free (temp);
22         (stack->count)--;
23     } // else
24     return dataOutPtr;
25 } // popStack
```


PROGRAM 3-10 Retrieve Stack Top (*continued*)

```
8 void* stackTop (STACK* stack)
9 {
10 // Statements
11     if (stack->count == 0)
12         return NULL;
13     else
14         return stack->top->dataPtr;
15 } // stackTop
```

PROGRAM 3-11 Empty Stack

```
1  /* ===== emptyStack =====
2      This function determines if a stack is empty.
3      Pre  stack is pointer to a stack
4      Post returns 1 if empty; 0 if data in stack
5  */
6  bool emptyStack (STACK* stack)
7  {
8      // Statements
9      return (stack->count == 0);
10 } // emptyStack
```

PROGRAM 3-12 Full Stack

```
1  /* ===== fullStack =====
2   This function determines if a stack is full.
3
4   Full is defined as heap full.
5   Pre    stack is pointer to a stack head node
6   Return true if heap full
7          false if heap has room
8  */
9  bool fullStack (STACK* stack)
10 {
11     // Local Definitions
12     STACK_NODE* temp;
13
14     // Statements
15     if ((temp =
16         (STACK_NODE*)malloc (sizeof(*(stack->top))))
17     {
18         free (temp);
19         return false;
20     } // if
21
22     // malloc failed
23     return true;
24 } // fullStack
```

PROGRAM 3-13 Stack Count

```
1  /* ===== stackCount =====  
2     Returns number of elements in stack.  
3     Pre  stack is a pointer to the stack  
4     Post count returned  
5  */  
6  int stackCount (STACK* stack)  
7  {  
8  // Statements  
9     return stack->count;  
10 } // stackCount
```

PROGRAM 3-14 Destroy Stack

```
1  /* ===== destroyStack =====
2      This function releases all nodes to the heap.
3      Pre   A stack
4      Post  returns null pointer
5  */
6  STACK* destroyStack (STACK* stack)
7  {
8      // Local Definitions
9      STACK_NODE* temp;
10
11     // Statements
12     if (stack)
13     {
14         // Delete all nodes in stack
15         while (stack->top != NULL)
16         {
17             // Delete data entry
18             free (stack->top->dataPtr);
19
20             temp = stack->top;
21             stack->top = stack->top->link;
22             free (temp);
23         } // while
24
25         // Stack now empty. Destroy stack head node.
26         free (stack);
27     } // if stack
28     return NULL;
29 }
```

Applications of Stack

- **Arithmetic Expression Evaluation:** Calculators use a stack structure to hold values for calculation.
- **Syntax Parsing:** Many compilers use a stack for parsing the syntax of expressions before translating into low level code.
- **Solving Search Problem**
- **Runtime Memory Management:** Almost all computer runtime memory environments use a special stack (the “call stack”) to hold information about procedure/function calling and nesting in order to switch contexts.

Types of Arithmetic Expression

- **Infix Expression**
 - Operators are placed between its two operands.
 - Example: $2 + 4$, $a - b / c$
- **Prefix Expression(Polish Notation)**
 - Operators are placed before its two operands.
 - Example: $+ 2 4$, $- a / b c$
- **Postfix Expression (Reverse Polish Notation or RPN)**
 - Operators are placed after its two operands.
 - Example: $2 4 +$, $a b c / -$

Algorithm: Infix-to-Postfix (Q, P)

Here Q is an arithmetic expression in infix notation and this algorithm generates

the postfix expression P using stack.

1. Scan the infix expression Q from left to right.
2. Initialize an empty stack.
3. Repeat step 4 to 5 until all characters in Q are scanned.
4. If the scanned character is an operand, add it to P.
5. If the scanned character is an operator Φ , then
 - (a) If stack is empty, push Φ to the stack.
 - (b) Otherwise repeatedly pop from stack and add to P each operator which
has the same or higher precedence than Φ .
 - (c) Push Φ to the stack.

- 6. If scanned character is a left parenthesis “(“, then push it to stack.**
- 7. If scanned character is a right parenthesis “)”, then**
 - (a) Repeatedly pop from stack and add to P each operator until “(“ is encountered.**
 - (b) Remove “(“ from stack.**
- 8. If all the characters are scanned and stack is not empty, then**
 - (a) Repeatedly pop the stack and add to P each operator until the stack is empty.**
- 9. Exit.**

Example: Q: $5 * (6 + 2) - 12 / 4$ and P: ?

Infix Expression Q	Stack	Postfix Expression P
5		5
*	*	5
(	* (	5
6	* (	5, 6
+	* (+	5, 6
2	* (+	5, 6, 2
)	*	5, 6, 2, +
-	-	5, 6, 2, +, *
12	-	5, 6, 2, +, *, 12
/	- /	5, 6, 2, +, *, 12
4	- /	5, 6, 2, +, *, 12, 4
	-	5, 6, 2, +, *, 12, 4, /
		5, 6, 2, +, *, 12, 4, /, -

Postfix Expression P : 5, 6, 2, +, *, 12, 4, /, -

Example: Q: $A * ((B + C) - D) / E$ and P: ?

Infix Expression Q	Stack	Postfix Expression P
A		A
*	*	A
(	* (	A
(	* ((	A
B	* ((	A B
+	* ((+	A B
C	* ((+	A B C
)	* (	A B C +
-	* (-	A B C +
D	* (-	A B C + D
)	*	A B C + D -
/	/	A B C + D - *
E	/	A B C + D - * E
		A B C + D - * E /

Postfix Expression P : A B C + D - * E /

Algorithm: Postfix-Evaluation (P, Value)

Here P is an arithmetic expression in postfix notation and this algorithm finds the value of this expression using stack.

1. Scan the postfix expression P from left to right.
2. Initialize an empty stack.
3. Repeat step 4 to 5 until all characters in P are scanned.
4. If the scanned character is an operand, push it to the stack.
5. If the scanned character is an operator Φ , then
 - (a) Remove two top elements of stack where A is the top element and B is the next-to-top element.
 - (b) Evaluate $T = B \Phi A$ and push T to the stack.
6. Pop the stack and assign the top element of the stack to Value.
7. Exit

Example: P : 5, 6, 2, +, *, 12, 4, /, - and Value: ?

Postfix Expression Q	Stack
5	5
6	5, 6
2	5, 6, 2
+	5, 8
*	40
12	40, 12
4	40, 12, 4
/	40, 3
-	37

Value:37

Review Questions

- ❑ Use a stack to convert an infix expression (e.g., $3 + 5 * (2 - 8)$) to postfix (3 5 2 8 - * +)? Show each step in translating this infix expression to postfix. Evaluate the resultant postfix expression using a stack?
- ❑ Suppose a stack is implemented using a singly linked list. The top of the stack is represented by the pointer TOP. Write the following functions in C/C++ or, their logic in psuedocode style:
int peek(struct Stack* stack);
int isEmpty(struct Stack* stack);
int getCapacity(struct Stack* stack);
int pop(struct Stack* stack);
- ❑ Compare space efficiency of stack using array vs linked list. When is one better than the other?
- ❑ Given an expression with parentheses, how does a stack help in checking its balance? Why can't we use a queue instead?
- ❑ If a stack is implemented using a fixed-size array, how do you handle overflow and underflow conditions?